Final Project

Applied Machine Learning with scikit-learn

Made by Alexander Nikolaychuk

Resources

Github Repository: Here

Google Colab Notebook: Here

Research Paper: Here

01

Dataset Exploration

2 datasets from Kaggle, and what they mean . . .

Overview

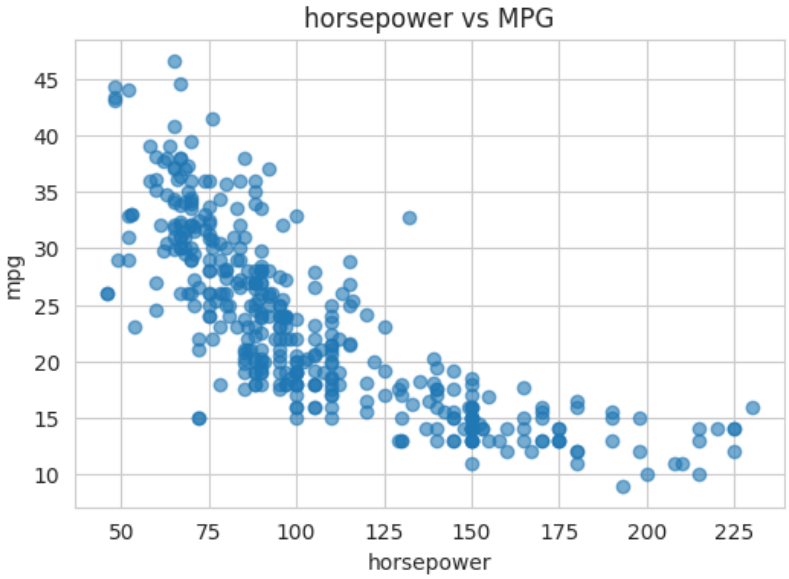
- Description of datasets
- Preprocessing
- Article introduction

Auto Mpg Dataset

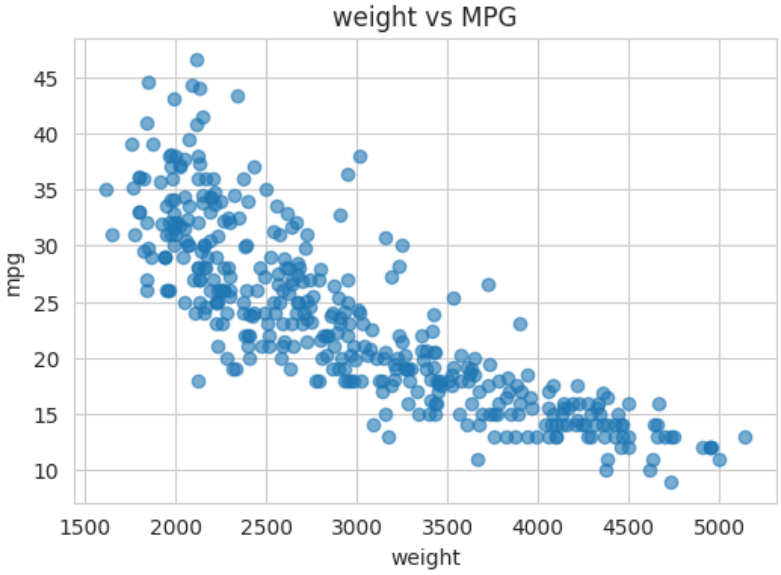
The first dataset used in this project is the Auto MPG Dataset from the UC Irvine Machine Learning Repository, accessed via Kaggle for consistency with class workflow. This dataset contains records of automobiles produced between 1970 and 1982, where each row represents an instance of a single vehicle. The target variable for this dataset is miles per gallon, or mpg. This numerical output makes it a regression task. Features include cylinders, engine displacement, horsepower, weight, acceleration, and model year, along with a categorical variable for 'country of origin.' Additionally, the dataset originally includes a "car name" field, which is a text feature combining manufacturer and model information. Overall, the dataset is moderately structured, consisting primarily of numerical data with some categorical and text components. However, preprocessing is required due to inconsistent data types (like how horsepower is stored as text instead of a float as missing values are represented by "?"), as well as the need to slice up the car name field to separate the model from the maker. I selected this dataset as its inconsistencies provide a good chance to practice data preprocessing as well as regression modeling.

Auto Mpg Features Graphed in Relation to MPG

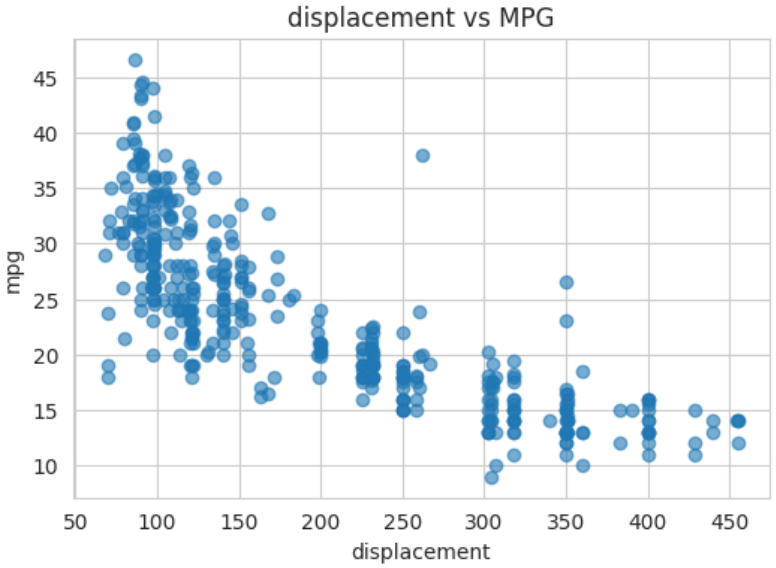
HORSEPOWER



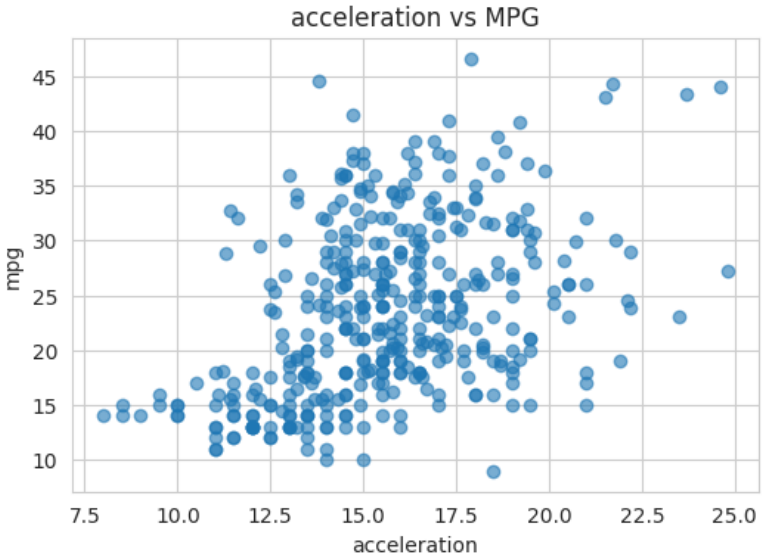
WEIGHT



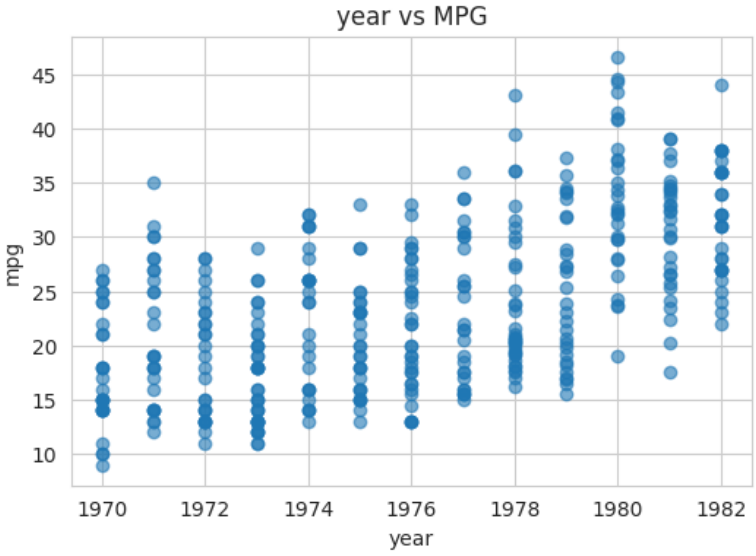
DISPLACEMENT



ACCELERATION



YEAR



As you can see, this dataset has clear (mostly negative) correlation between its features. As weight, engine size, and power go up, and fuel efficiency goes down.

Intro to Research Paper - Bandırma Onyedi Eylül University (Turkey)

Authors: **Alpay Doruk & Muhammed Ali Bayram**

Year published: **2023**

Their approach included models such as

- *Decision Trees*
- Random Forests
- Support Vector Regressors
- *Neural network-based methods (LSTM and GRU)*

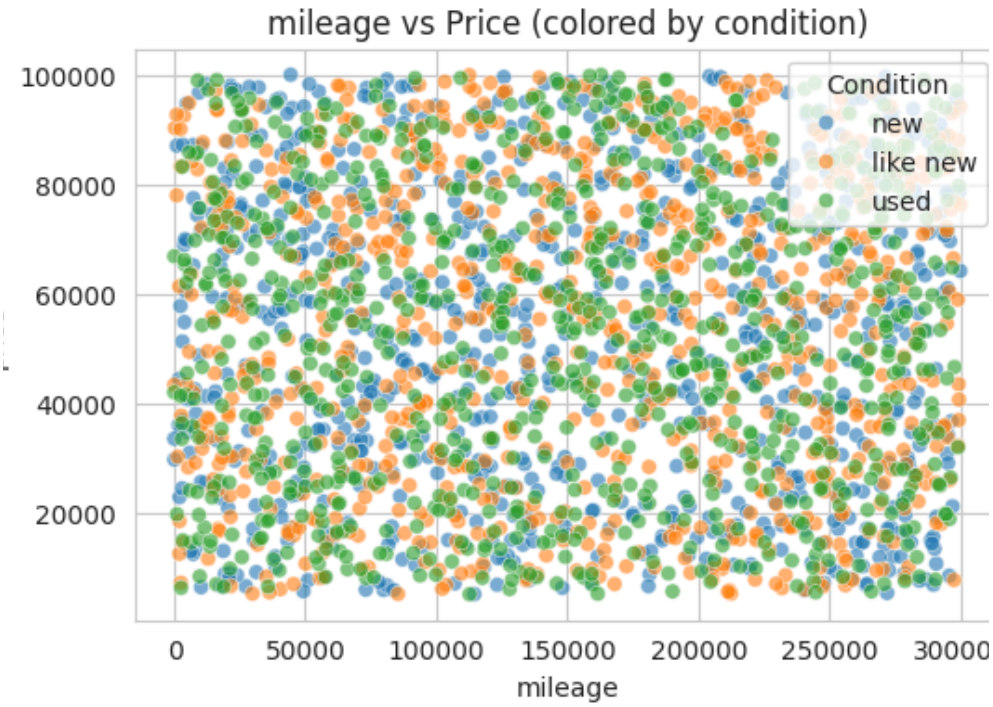
To introduce this research paper shortly, this study by Doruk and Bayram applied multiple machine learning models to the Auto MPG dataset and found that ensemble methods such as Random Forest achieved the best performance.

Car Price Prediction Dataset

The second dataset I will be using is the Car Price Prediction Dataset. Also obtained from Kaggle, this dataset focuses on modern vehicle listings (2000-2022) and their market prices. Each row represents a single car listing, and the target variable is the vehicle price, also making this a regression problem. The features include a mix of categorical and numerical variables such as brand, model, year, transmission type, mileage, and condition. Compared to the Auto MPG dataset, this dataset has a more clearly defined structure, with cleaner formatting (maker and model are already separate). The data types include numerical fields (year, mileage, price) and categorical fields (brand, transmission, condition). Although it is more structured, it still requires preprocessing, particularly for encoding and preparing the data for modeling. Lastly, our target variable for this dataset is price.

Car Price Features Graphed in Relation to Price

MILAGE



YEAR

This dataset has no correlation between mileage, condition, and price. While this normally be thrown out, since we are in an academic setting, I have decided to see what happens if we train models on very poor data.

Data Preprocessing

Full disclosure: These datasets are reused from my database project; I just ran a query to select all attributes in my whole database and exported as csv, and uploaded it to google drive. I will still explain the further prepossessing I had to do for my database project.

With the original datasets, the first thing that I did with the data was clean it up. I will explain why this was necessary through an example. When combining both tables, we ended up with these (and more) manufactures:

- Chevrloet
- Chevrolet
- Chevy
- Mercedes
- Mercedes-Benz
- Subaru
- Sabaru

To provide a short summary of how I originally cleaned these datasets, it was done with a mapping in my SQL relational database.

```
INSERT INTO ManufacturerMapping VALUES
('chevy', 'Chevrolet'),
('chevrolet', 'Chevrolet'),
('vw', 'Volkswagen'),
('volkswagen', 'Volkswagen');
```

I then joined the two datasets, taking care to only select the first word of the model name from the mpg dataset in order to only grab manufacturers, and that cleaned up the manufacturers.

While there was much more to clean, the only other significant example I would like to mention is how years were stored in both datasets. In mpg, it is of the format (90) for 1990, while it would have been stored in the price dataset as (1990). This lead to this query:

```
UPDATE Vehicles
SET year = year + 1900
WHERE year < 100;
```

While it is simple, the impact is profound. With these three lines of code, I set all values from the autompg dataset to match the price dataset.

End of database project preprocessing.

For the code of this project, all the rest of the preprocessing happened inside the Pipeline and ColumnTransformer, which ensured it is applied consistently during training, testing, and GridSearchCV.

Missing Data

For my database, missing values were fine. However, in our case, rows with missing target values are removed first using `df.dropna(subset=['price'])`, also known as *listwise deletion.* This is because a supervised regression model cannot learn from rows where the answer it is trying to predict is missing. For feature columns, missing numerical values are handled with `SimpleImputer(strategy='median')`, which replaces missing values in each numeric column with that column's median value. The median was chosen as it is less affected by extreme outliers than the mean tends to be.

Just to note, Imputation was done after the train/test split. This is crucial because this prevents data leakage, as the median imputation value differs if all of the data is used, versus if only the training data. If the median imputation is done on all of the data, then the model inadvertently gets access to data that is supposed to be reserved for testing, also known as data leakage.

Scaling / Standardization

For my code, I made numerical features standardized using StandardScaler(). This makes numeric features have a mean of 0 and a standard deviation of 1. This is important as features like year and mileage may have very different numeric ranges, so mileage would be a larger-scale variable and would dominate the model, instead of mileage and year being both used evenly to predict. This is useful for models like Linear Regression and MLPs because they are sensitive to feature scale.

Note: This differs from normalization. Scaling scales numerical features to a mean of 0 with a standard deviation of 1, while Normalization Rescales values to a fixed range (usually 0–1). This means normalization forces everything into a strict [0,1] range, with

outliers skewing the curve or not being recorded properly. While this may make normalization seem inferior, it is still used for other models, such as Distance Based KNN.

Encoding

As you might already know, encoding converts categorical (like text) data into numerical format so machine learning models can process it. Categorical columns are converted into numeric format using `OneHotEncoder(handle_unknown='ignore')`. This creates separate binary columns for each category. For example, if a column like condition_name had values such as "excellent", "good", and "fair", one-hot encoding would create separate columns for each condition with 1s and 0s indicating whether each row belongs to that category. The handle_unknown='ignore' setting prevents errors if the model sees a new category in the test set that was not present during training.

02 Model Development with scikit-learn

3 models from scikit-learn, and how they work . . .

Overview

- Models I chose
- How they were trained
- Training split/cross validation

My project is going to be applying 3 models to predict mpg, as well as price. Since this will be a supervised learning task, I have decided to select the following three models:

Linear Regression

As I am sure many other students have done (and for good reason), I also have selected linear regression as a starting baseline model. Not only is it simple, but it has a great effectiveness for regression tasks where relationships between features and the target variable are approximately linear. Now given the graphs of the data we have seen earlier, the relationships in the Autompg dataset are not exactly linear, so it will make for an interesting comparison with the other two models I have chosen that could perhaps better capture the non-linear relationship. This model works by fitting a linear equation

that minimizes the error between predicted and actual values, making it easy to interpret how each feature contributes to the prediction. However, for the price dataset, which has noisy relationships, its performance may be limited. Overall, it serves as an important reference for comparing the more complex models.

Multi-Layer Perceptron (MLP)

The neural model I have chosen, or more specifically the Multi-Layer Perceptron (MLP), is capable of learning complex, nonlinear relationships between our features and target variables. In short, it consists of multiple layers of interconnected neurons that transform input data through weighted connections and activation functions. MLP was chosen because real-world data, especially pricing data, often involves nonlinear patterns that simpler models cannot capture. In this project, MLP is expected to perform well on the MPG dataset by capturing interactions between features, and it may also provide improvements for the price dataset if sufficient patterns exist, but that is highly unlikely.

Decision Tree

I chose to have a decision tree as the final model as they also can model nonlinear relationships, but they do so in a much different way. They use a series of rule-based splits, splitting the data into parts and assigning predictions within each region, making them good for structured data like vehicle specifications. As a bonus, they are also highly interpretable, as their decision process can be easily visualized. After all, we talked about trees all the way back in CSCI 41. For the MPG dataset, Decision Trees are expected to perform strongly due to their ability to capture threshold-based relationships. As for the price dataset, I am not sure what it will predict as it will struggle since the features do not provide clear splits.

Training split and cross validation

I decided to evaluate my models using both a 80/20 train/test split, as well as a 5-fold cross-validation. While I am sure you are familiar with this concept, I will still elaborate. First, the data was split so that 80% was used for training the model and 20% was held out as test data to evaluate final performance. In addition to this, a 5-fold cross-validation was applied on that 80% training data during model training, where four folds were used to train the model while one fold was used for validation, and this process happened five times, meaning that each fold served as the validation set once. The results were then averaged to produce a more robust estimate of model

performance, reducing the impact of variability from any single split and helping ensure that the model generalizes well to new data.

03 Hyperparameter Tuning and Model Comparison

How to improve model performance . . .

Overview

- Hyperparameters explained
- How they are tuned
- Model Effects

Before we tune hyperparameters, we need to understand what they mean.

For **Linear Regression**, we have only one hyperparameter:

- fit_intercept: [True, False]

This one parameter decides if the linear model/equation has a y-intercept. Going back to algebra, you may remember “y = mx + b.” If this parameter is set to false, then all we have is “y = x + b”.

MLP (Neural Network):

- hidden_layer_sizes: [(16,), (32,), (16, 8), (32, 16)]

This defines how the neural network is built. For each pair of (x,y), x is the amount of neurons, and y is the optional second layer. (16, 8) is 2 layers, 1st layer being 16 neurons and the second being 8 neurons.

- activation: ['relu', 'tanh']

This is the activation function, will affect how neurons transform input. relu is the most common, being fast, and handling nonlinearity well. Meanwhile, tanh is smoother, and outputs between -1 and 1.

- alpha: [0.0001, 0.001, 0.01]

This is the penalty for large weights, in order to prevent overfitting. The higher it is, the more underfit the model will be. It is important however to not go overboard and create a model that is too general with a large prediction error.

- learning_rate_init: [0.0005, 0.001]

This controls how fast the model learns. The longer the model learns, the more stable it learns, but at the cost of computation. Our goal is to train the model quickly/easily without overshooting the optimal solution, causing the loss function to diverge (go to infinity).

Decision Tree:

- max_depth: [3, 5, 7, 10, None]

This is how deep the final tree is allowed to go, which determines the amount of splits/paths it may have. More branches results in a more complex model, which could either be good in that it captures complicated patterns, or it could end up fitting to the data too well and predict new data poorly.

- min_samples_split: [2, 5, 10, 20]

This is the minimum number of samples before a node splits. This has a similar depth to

max_depth, as increasing the minimum amount of data before the branches split is another way to reduce overfitting.

- min_samples_leaf: [1, 2, 5, 10]

Yet again this is similar to min_samples_split, but rather than being the minimum number of samples before a node split, it is the number before a leaf (child) is made.

Now that you have at least a simple understanding of the hyperparameters, here is what hyperparameters I used:

Using GridSearchCV:

Linear Regression:

- fit_intercept: [**True**, False]

MLP (Neural Network):

- hidden_layer_sizes: [(16,), (32,), (16, 8), (**32, 16**)]
- activation: [**'relu'**, 'tanh']
- alpha: [0.0001, **0.001**, 0.01]
- learning_rate_init: [0.0005, **0.001**]

Decision Tree:

- max_depth: [3, 5, **7**, 10, None]
- min_samples_split: [2, 5, **10**, 20]
- min_samples_leaf: [1, **2**, 5, 10]

To clarify, GridSearchCV is a tool in scikit-learn that automatically finds the best combination of hyperparameters for a model using cross-validation. The **Bold** in my list represents what parameter worked best for the models predicting mpg, and the underlined represents what worked best for the price prediction (Although it did not mean much for price prediction).

Results:

Overall, hyperparameter tuning had varying levels of impact across models. For Linear Regression, tuning had minimal effect as there was only one parameter, but that is to be expected from a simple baseline model. The Decision Tree model showed noticeable improvement when controlling tree depth and split criteria, which helped reduce overfitting and improved generalization. The MLP model demonstrated the most sensitivity to hyperparameters, with performance improving when using larger hidden layer configurations and appropriate regularization. This highlights the importance of careful tuning for neural networks, as their performance can vary significantly depending on parameter choices.

In terms of model comparison, performance differed across datasets. For the MPG dataset, all models performed relatively well, with Decision Trees and Linear Regression achieving strong results due to clear relationships between features such as weight and horsepower. In contrast, for the price dataset, all models performed poorly, with similar error metrics across Linear Regression, Decision Tree, and MLP. This confirms that the available features did not provide sufficient predictive power, and that model choice had less impact than data quality. Overall, the results show that while hyperparameter tuning can improve performance, the effectiveness of a model is highly dependent on the underlying structure and quality of the dataset.

04 Results, Evaluation, and Literature Comparison

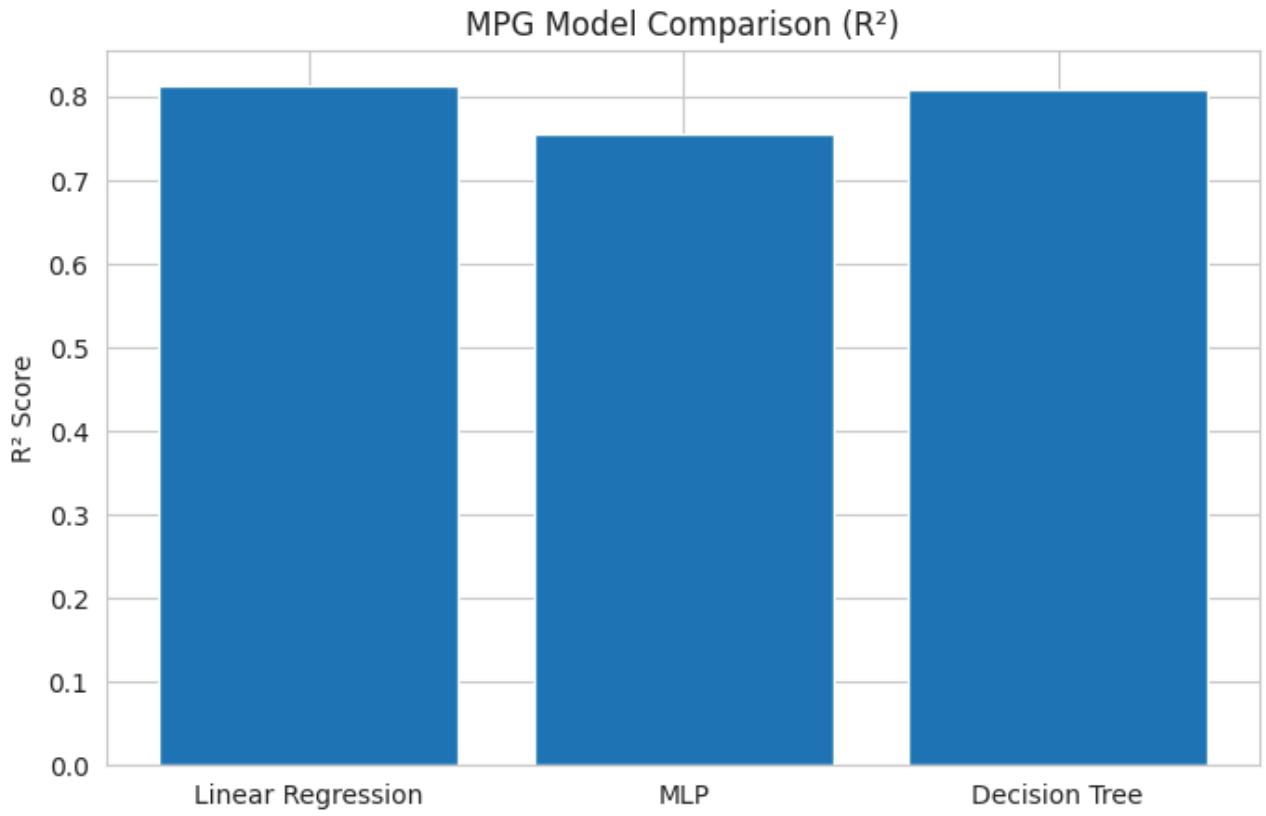
What it all means . . .

Overview

- Metrics RMSE and R2
- Residual plots and their distribution
- Model Comparison
- Previously cited work

Model Error Comparison

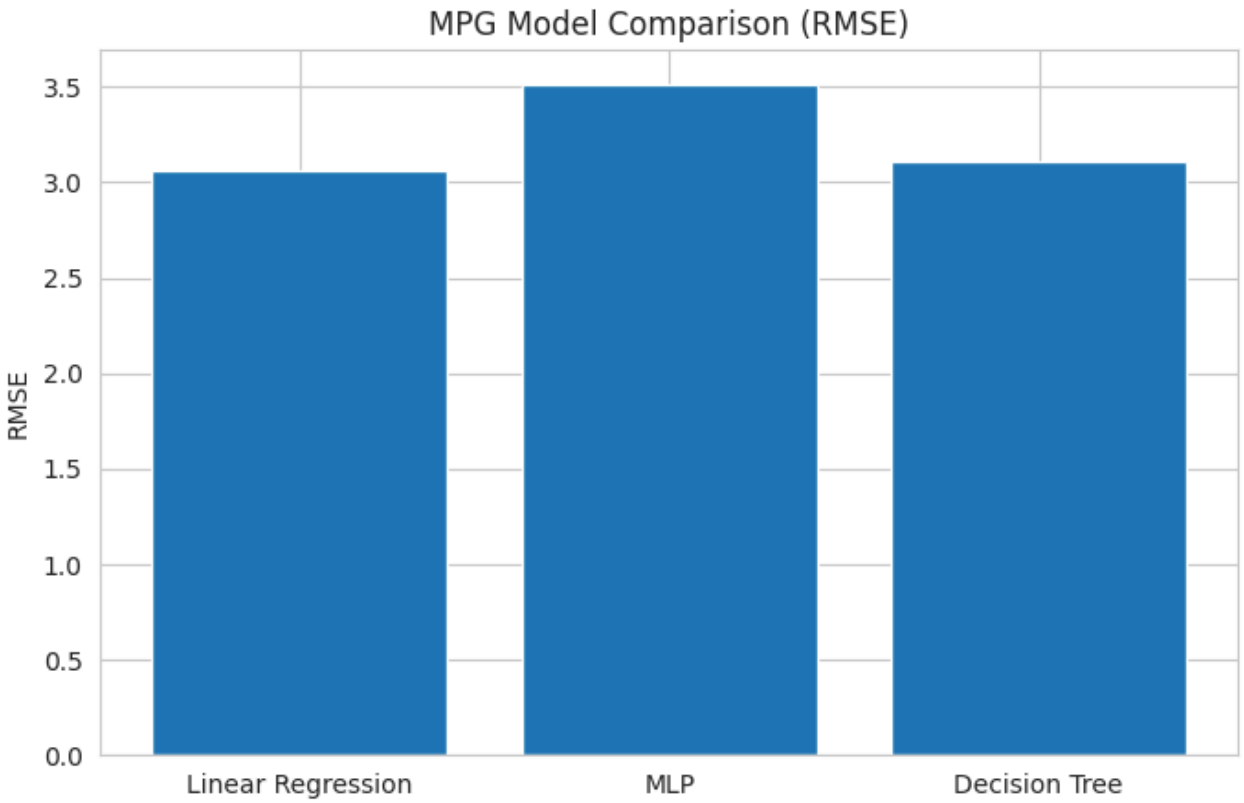
Now that we have successfully trained our models, lets see how they did:



First, it is important to understand that the R^2 error measures the amount of variance in the model's predicted values, with values of 0 meaning the model is as good as just guessing the average, and values closer to 1 indicating better performance.

With that being said, looking at the chart above, we can see that for predicting **mpg**, the MLP performed the worst, with Linear Regression being very close to the Decision tree model. Here are the actual values, in the same order as the charts:

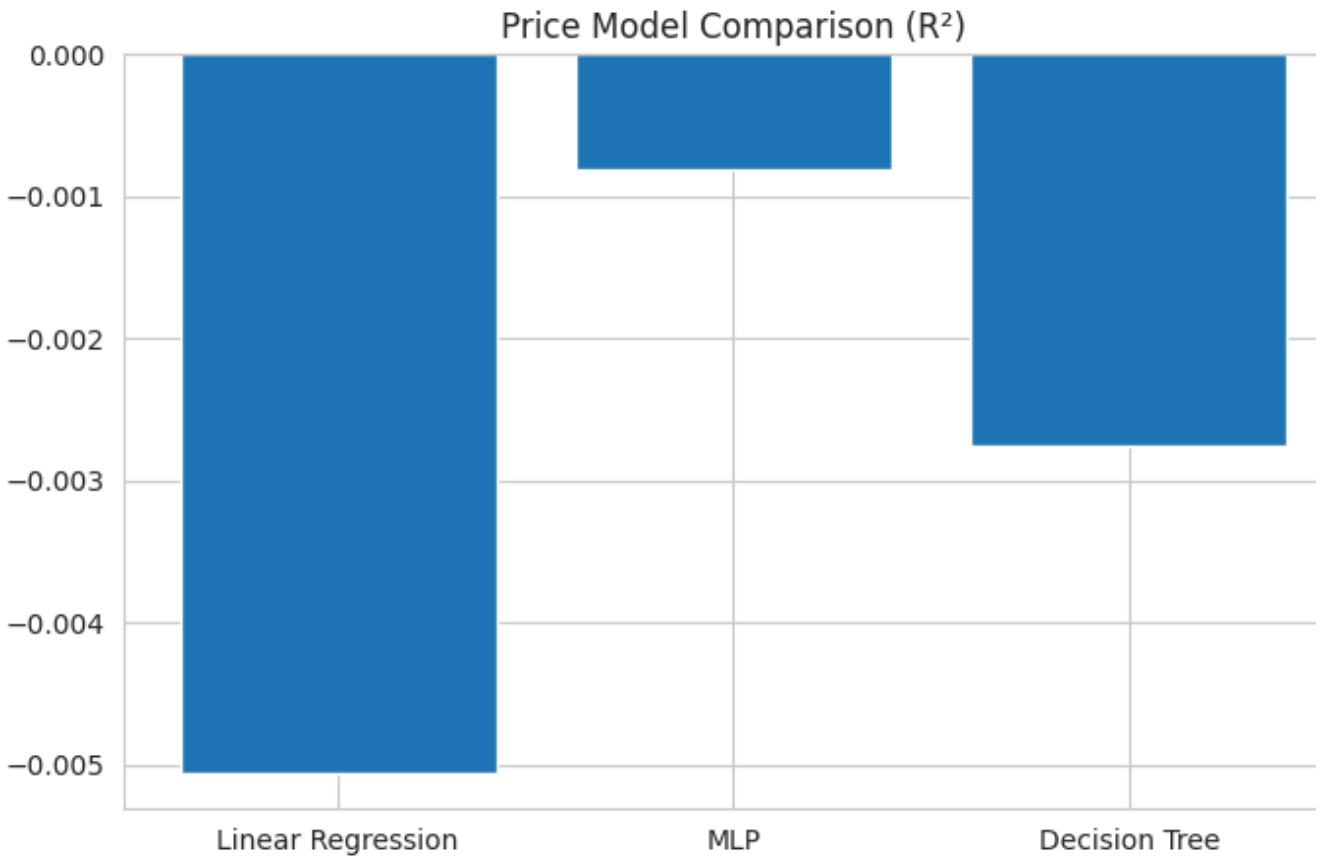
```
[0.8139925231055446, 0.7553302556670074, 0.8086983718165786]
```



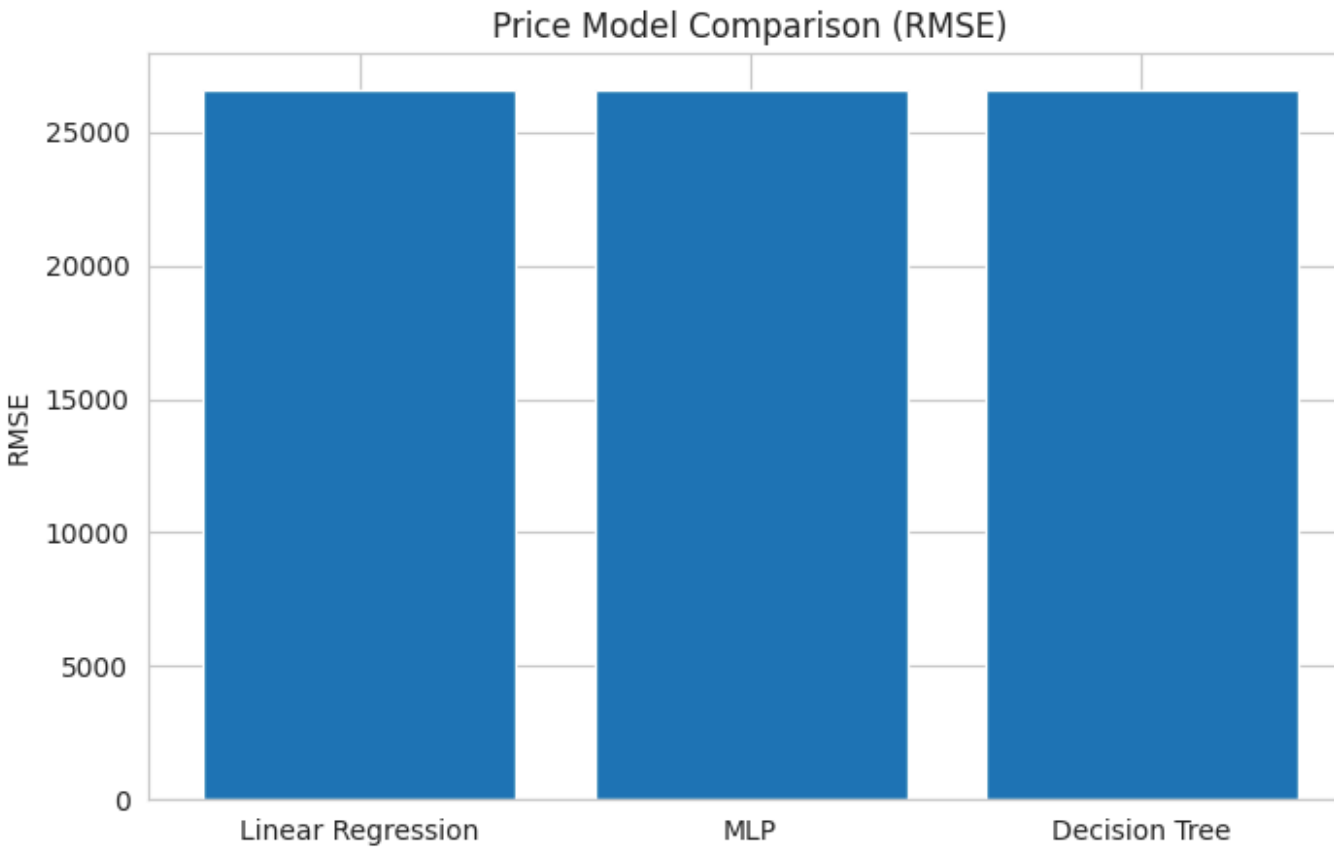
For another metric of comparison, we have RMSE (Root Mean Squared Error). This is calculated by taking the average of the squared error, and then taking the square root. The reason this metric is used is because when the errors are squared, larger errors are exacerbated, even when the square root is taken later. If you look at the precise values below, it shows that the predictions for each of the three models were off by about 3 mpg.

All in all, this graph shows similar results to the R^2 graph. MLP surprisingly did worse than the other two models, and even more surprisingly linear regression had the least amount of error.

```
[3.066078166069276, 3.516480032131276, 3.1094054452447595]
```



Given earlier that values of 0 means that the model is as good as just guessing the average, a negative R^2 means that the models are even worse than guessing the average. This shocked me, as guessing the average for a dataset such as the one I found is also terrible, given how the data is all over the place. The last thing I would like to point out is that MLP actually performed the best, possibly being the closest to finding a trend in the data, but as you can see below with the RMSE, that does not seem to be true.

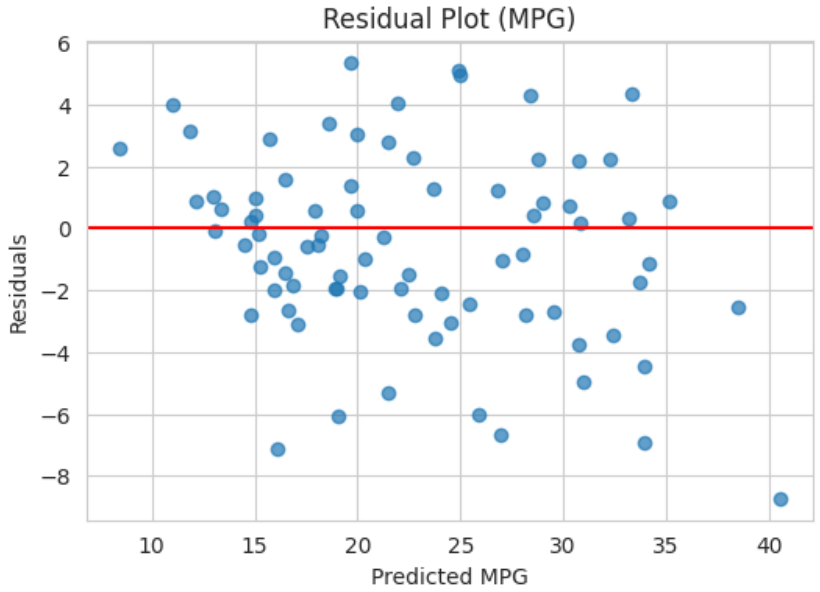


It is without a doubt that the models performed poorly, with each of them getting an average error of about $25,000 per car. And to assure you that this graph is not an error, I have pasted the RMSE's of each model below, and yet again we have an interesting observation. With a dataset as poor as the one I have chosen, all simple models do equally terrible.

```
[26598.630479163316, 26542.365205198876, 26568.181813338946]
```

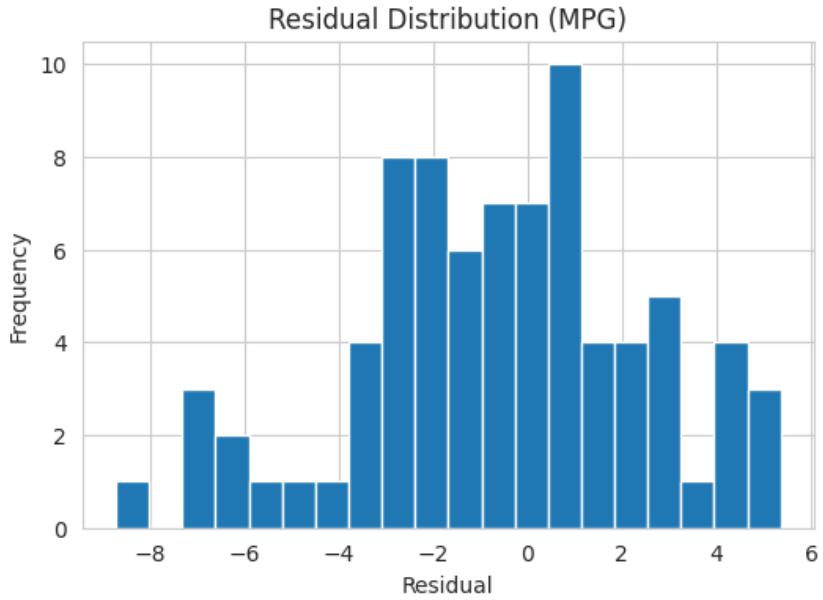
Model Residual Data

Below I have two diagrams relating to my linear regression model,with the first being a residual plot. This plot shows my models predicted values versus the residuals (actual – predicted).



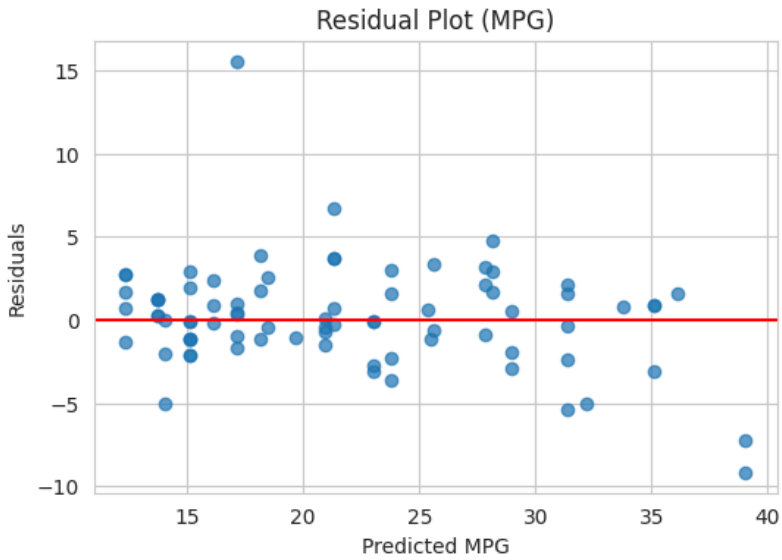
A good residual plot shows points randomly scattered, centered around 0, with no clear pattern. That shows the model is not biased, and is capturing relationships well.

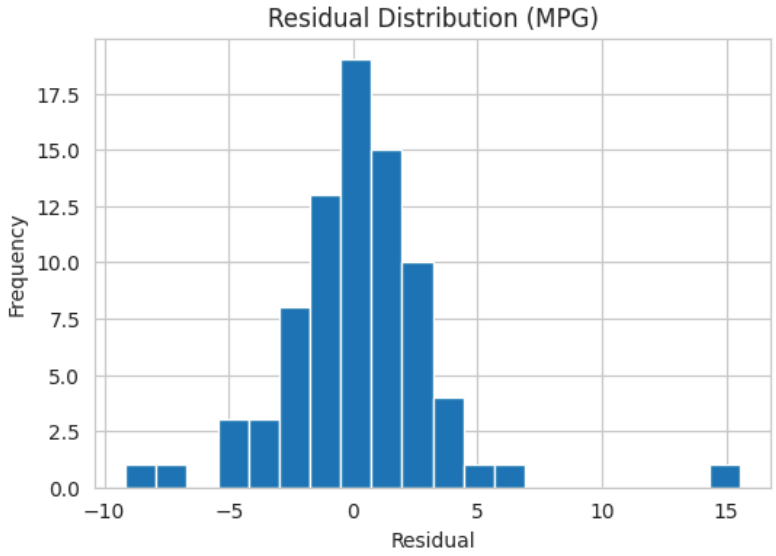
As we can see, that is mostly true, with the model overpredicting a bit (you can see how the graph ranged from 6 mpg over to 8 mpg under.)



Above, we have a residual distribution graph. This shows how the residuals are distributed, and should be centered around 0 and roughly symmetric (preferably bell-shaped). For my linear regression model, the distribution is just a bit left-skewed, agreeing with the first graph in that the model is overpredicting.

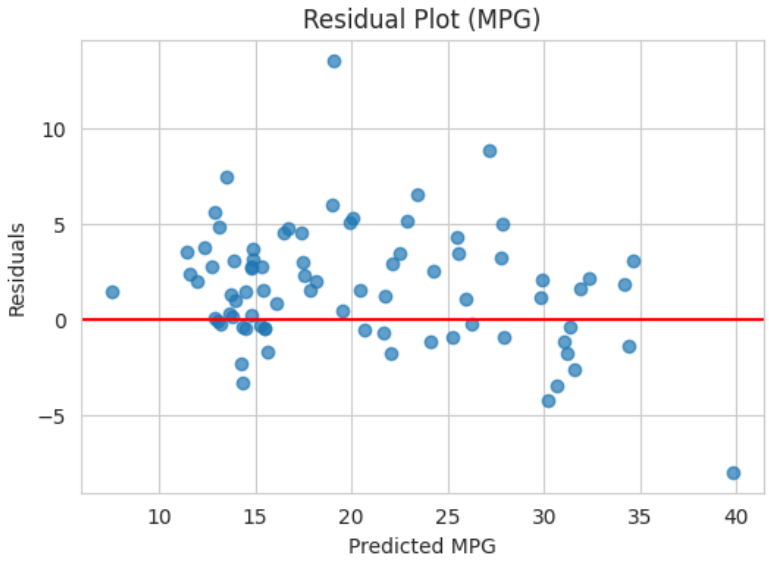
MLP Predicting MPG

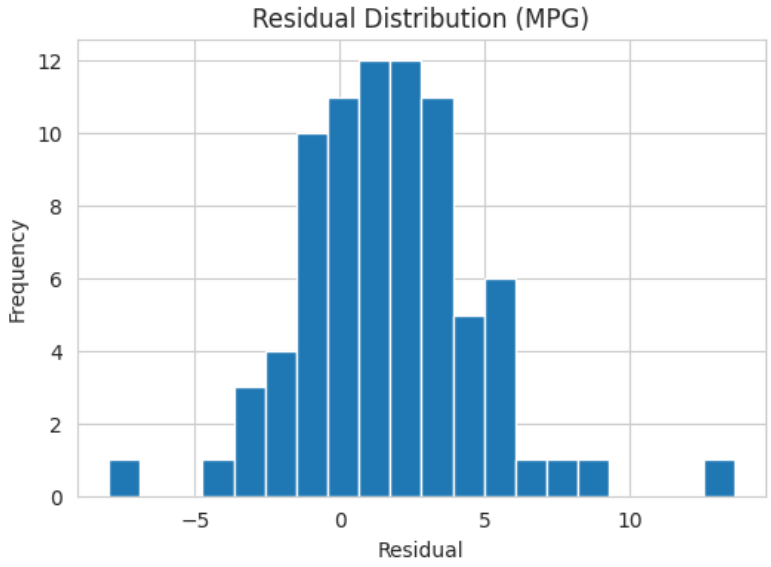




Here, we see that the data for the residual plot is much closer to the line (0), as well as an overall neater bell curve. However, it seems that when the model is wrong, it is wrong by a lot more, which is why it's R^2 score was lower and it's RMSE was higher. If you recall, R^2 and RMSE penalize larger errors the most. So while at first the MLP might have seemed to be the worst, it in fact is the best if you remove the outliers.

Decision Tree Predicting MPG



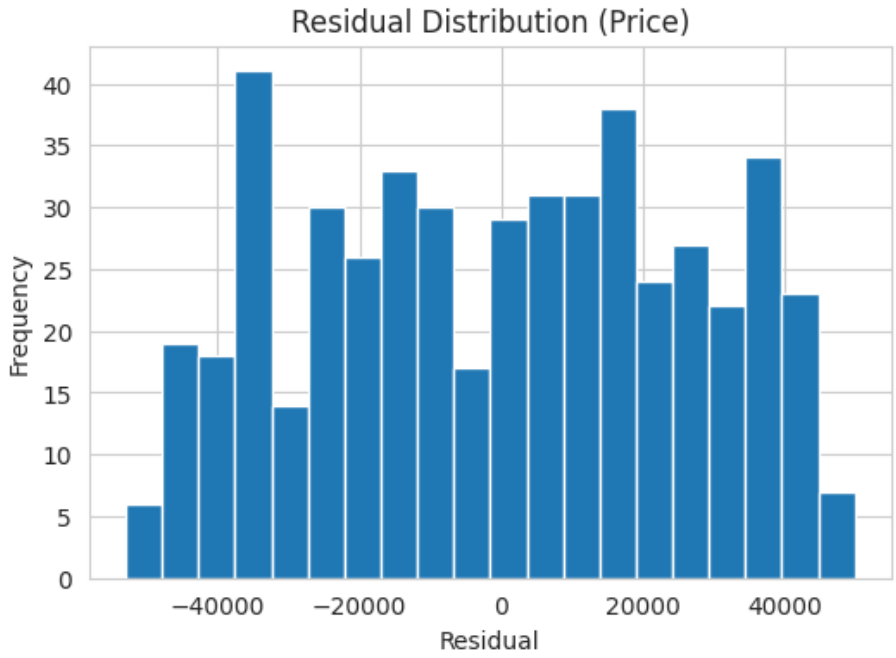


Lastly for the MPG dataset, we have our decision tree model. Unlike the linear regression, we can see this model is underpredicting mpg. Overall, as seen by the residual distribution, it is a good balance to my other two models, providing a diverse contrast on how each model solves the same problem.

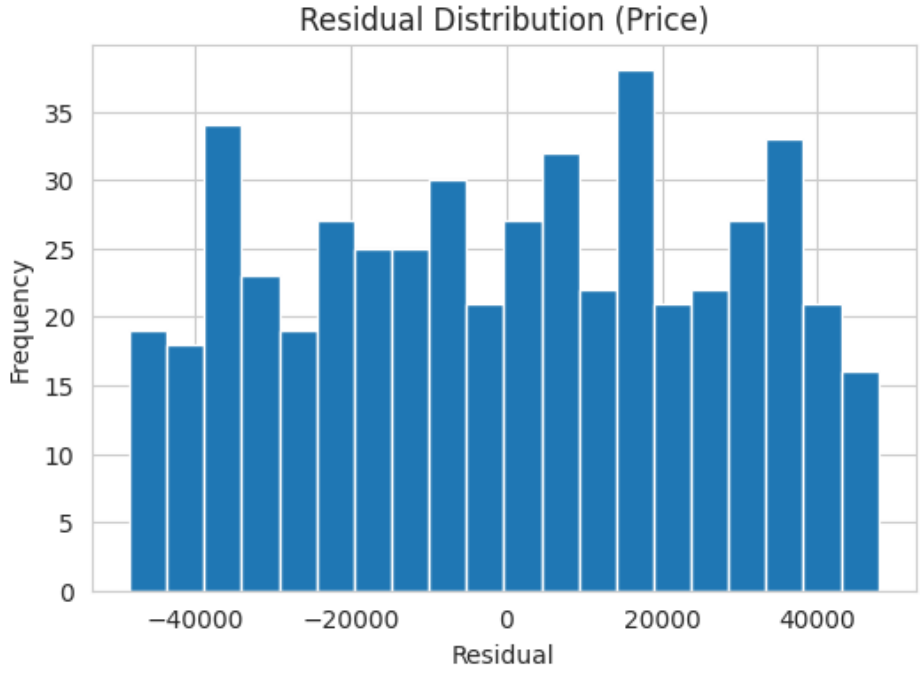
Price Residual Data

Below are the plotted residuals for my models predicting price. As expected, they did not do well. While I will not comment on all of them individually as they all mostly performed the same, there is an interesting observation that we can point out.

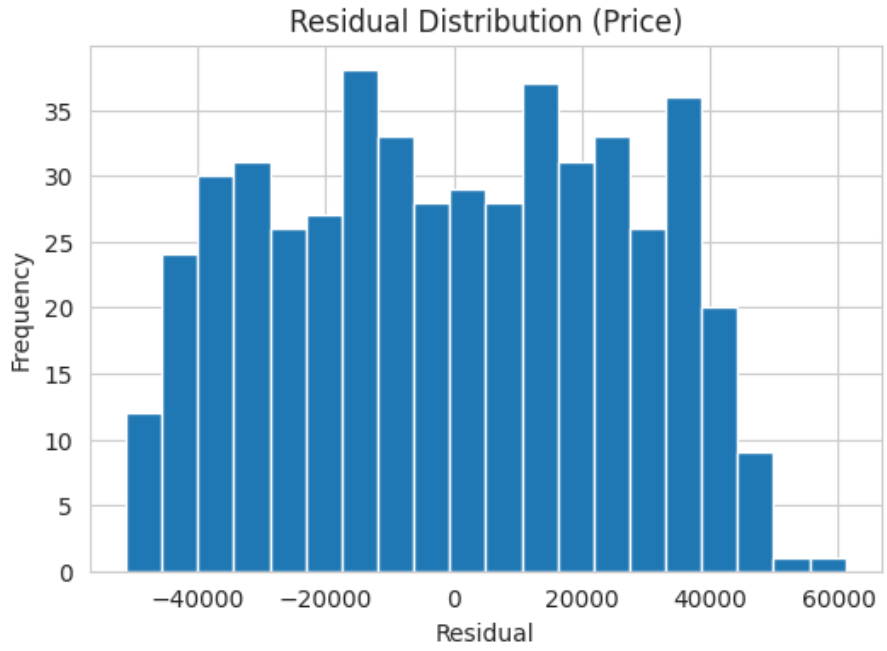
Linear Model Predicting Price



MLP Predicting Price



Decision Tree Predicting Price



While linear regression and MLP both had the same wild variation on predicting price, the decision tree did not. As you can see, it has these vertical lines. This is a great insight into how decision trees work.

A decision tree splits the data into groups based on rules like mileage < 50,000. It keeps splitting into smaller groups until at the end of the branch, it assigns a prediction, which is the average value of that group. Decision trees do not predict smoothly, and with bad data, they cannot define good rules to distinguish between cars, and end up predicting the same value repeatedly.

Research Article ← hyperlink

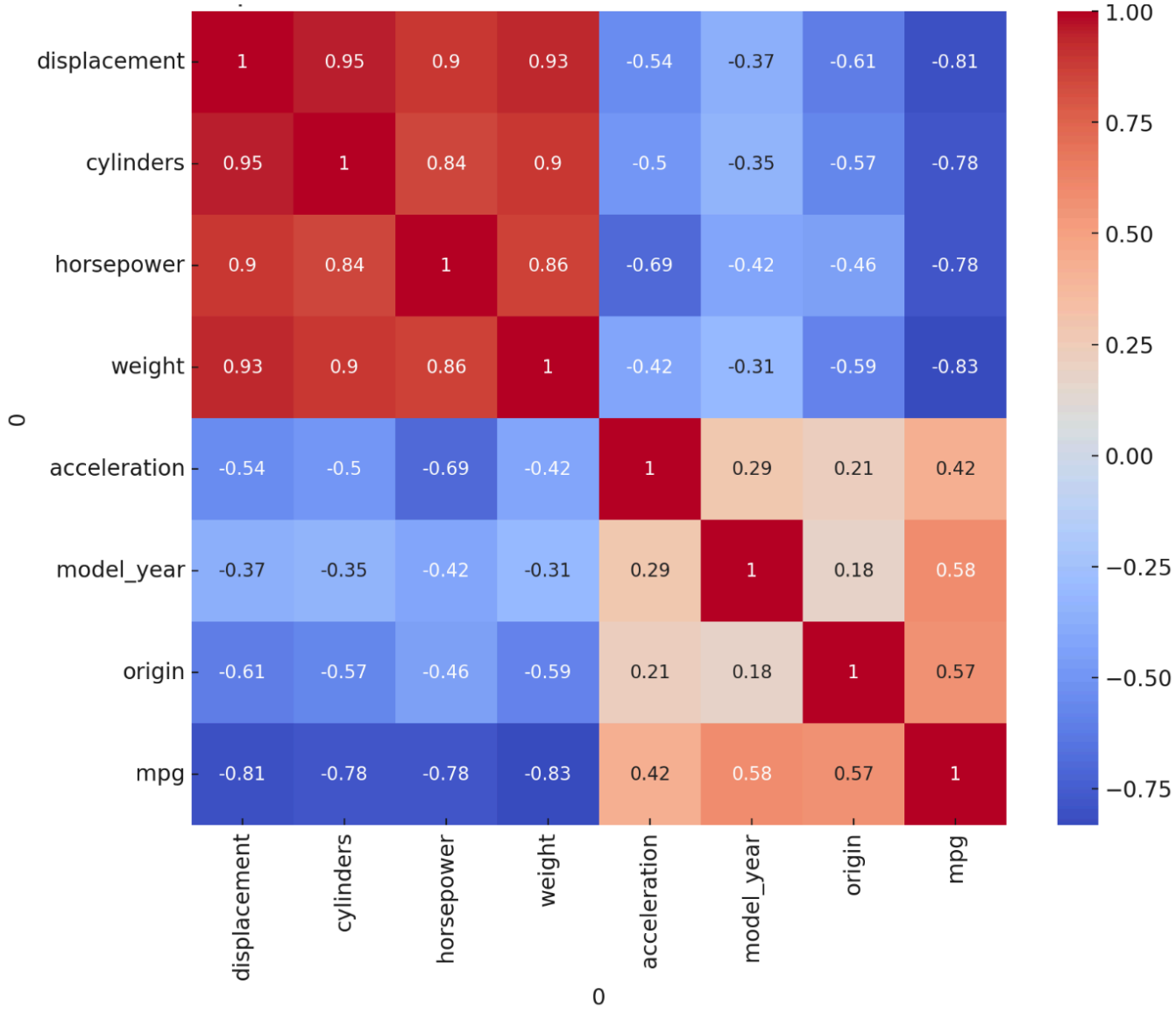
Intro to Research Paper - Bandırma Onyedi Eylül University (Turkey)

Authors: **Alpay Doruk & Muhammed Ali Bayram**

Year published: **2023**

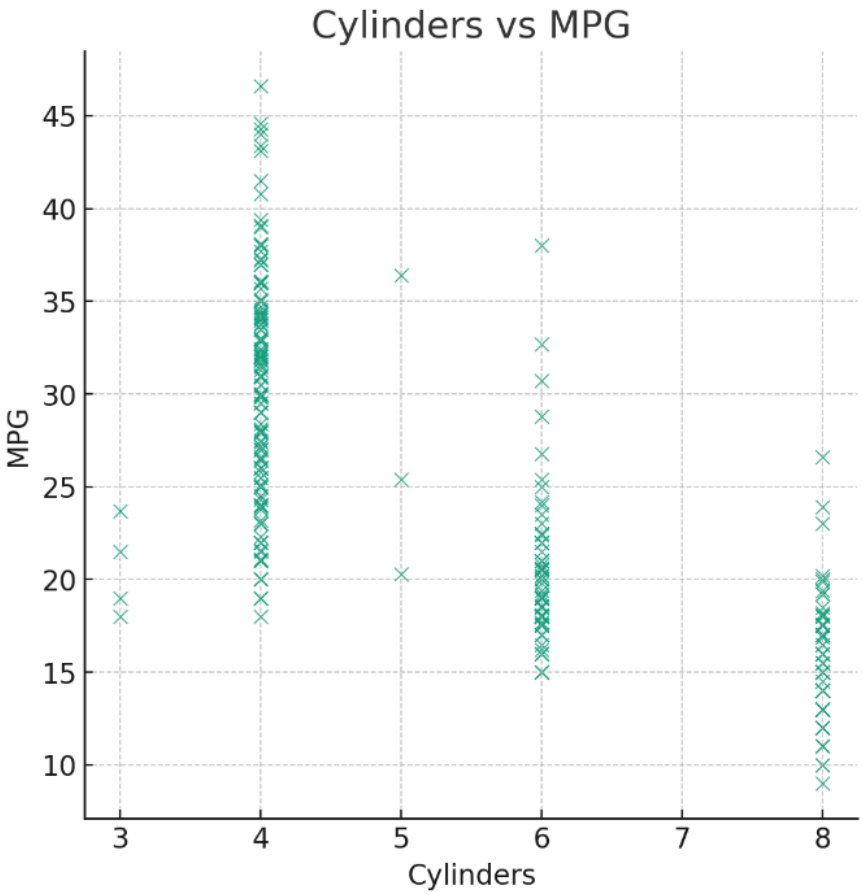
For my research paper, I found Doruk and Bayram of Bandırma Onyedi Eylül University of Turkey. Their paper is based on the auto mpg dataset, where they used similar models to me. That included decision trees, support vector regression, neural networks (LSTM and GRU), and random forest models. The models were evaluated using 10-fold cross-validation as opposed to my 5-fold cross validation, with the Random Forest Regressor achieving the best performance ($R^2 \approx 0.77$ and lowest RMSE). In comparison, for my project, comparable performance was achieved using simpler models, with Linear Regression and Decision Trees reaching R^2 values above 0.80 on the test set.

This similarity in performance is likely due to the strong relationships present in the Auto MPG dataset, particularly between features such as weight, horsepower, and displacement, which were also highlighted in the paper's correlation analysis; see Figure 1 from their paper:



This shows that the relationships within the Auto MPG dataset can be effectively captured using both linear and nonlinear methods. However, in contrast to the MPG dataset, all models performed poorly on the price dataset, indicating that the available features of the dataset were insufficient.

Differences in our results may be attributed to preprocessing choices, including feature scaling, encoding, and pipeline-based handling of missing data, which they did not delve into in their paper. Additionally, the paper used more complex models such as Random Forest, which can better capture nonlinear relationships through ensemble learning. Lastly, I would like to point out something they have included in their paper that I did not:



I have not thought to graph the relation of cylinder to mpg, but as it turns out, it also has a negative/inverse relationship, similar to engine displacement, horsepower, and weight. To finish this section, I highly recommend you go read their paper for yourself as there is much more that I have not covered.

Conclusion

To bring some final excitement to my project, I have decided to list out these two stats:

BEST MODEL:

Decision tree (But linear was a close second)

BEST DATASET:

Auto MPG; it had real learnable relationships

Overall, this project demonstrates the ability of the models I used to capture relationships within the data where there is one. Linear Regression provided a simple and interpretable baseline, while Decision Trees and MLP models were able to capture more complex and nonnonlinear patterns. On the other hand, the models also have limitations. Linear Regression is restricted to linear relationships, Decision Trees can overfit or produce overly simplified predictions, and MLP models require careful tuning and are sensitive to hyperparameters. Additionally, all models performed poorly on the price dataset, highlighting a key limitation: model performance is highly dependent on the quality and relevance of the available features. To apply my research to the real world, the MPG models could show practicality in estimating fuel efficiency during car manufacturing, as brands would be able to make predictions of new cars before they ever needed to build the first one.

For my final words, I would like to say if I ever did this project again, one thing I would like to improve on is that I would include a dataset for categorical prediction and the corresponding models. That is all, thank you for reading.